



• 2025 •

TESTING POLICY DOCUMENT

GAMIFIED FINANCIAL VISUALIZER

For

Demo 4

Presented by :



CodeBlooded



Introduction	
Project Overview	
Testing	
Unit Testing policy	
Integration Testing policy	
E2E testing policy	
Testing workflow	
Quality Requirements testing	
Performance	
Availability	



Availability with Uptime Kuma	
Usability	



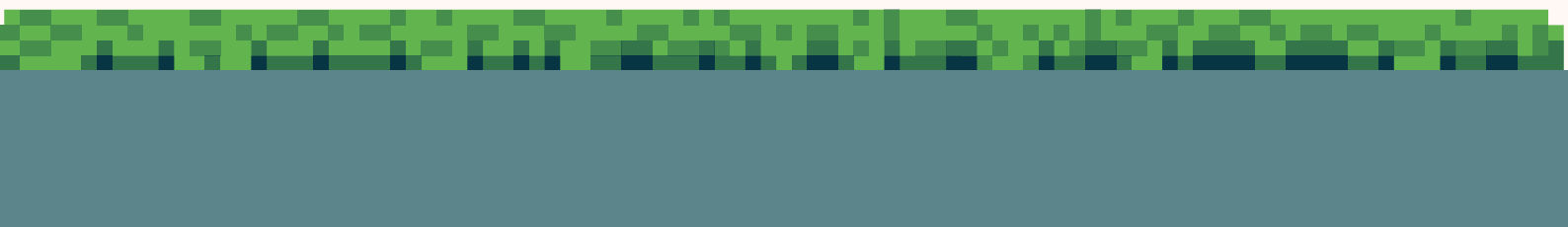
Introduction

The Gamified Financial Visualizer is an interactive platform designed to help users manage and visualize their finances engagingly and educationally while promoting financial literacy. This document outlines the architectural design of the system, with a focus on key quality attributes including performance, usability, security, reliability, availability, scalability, maintainability, compatibility, and legal compliance. Additionally, it addresses the system's evolution from a traditional client-server model toward a scalable microservices architecture to support future enhancements.

This Testing Policy outlines the principles, scope, and methodology for validating the reliability, performance, and user experience of the gamified finance application. Designed to engage users through interactive financial challenges, real-time feedback, and personalized goal tracking, the app integrates modular backend services, secure authentication, and dynamic frontend components.

The purpose of this document is to ensure that all features—ranging from transaction simulations and leaderboard logic to theme persistence and risk engine triggers—are rigorously tested across development cycles. It establishes a consistent framework for unit, integration, end-to-end, and usability testing, enabling scalable growth and reviewer-friendly documentation.

By adhering to this policy, the development team commits to delivering a robust, intuitive, and secure platform that meets both technical standards and user expectations. Testing is not treated as a final checkpoint but as an embedded discipline throughout the lifecycle of every feature and service.



Project Overview

This project is being developed as part of a university capstone course and is scoped to balance innovation with practicality, targeting a functional and demonstrable product within academic timelines. The following elements define the scope of this system:

Financial Tracking & Visualization

- Users can upload their bank statements for automatic input and categorisation of their transactions.
- The system will process and visualize transactions using dynamic dashboards and 3D/AR environments.
- AR-based interaction allows users to view a financial "world" like "savings buildings" or "budget trees".

Gamification Features

- Reward systems for achieving financial goals.
- Points and achievements for completing educational modules and maintaining budget discipline.
- Community features such as leaderboards and challenges.

AI-Powered Financial Insights

- Transaction classification using NLP (DistilBERT).
- Personalized financial advice delivered through a conversational interface (local LLM such as LLaMA 3).
- Goal recommendations and behavioural tips based on user profiles.

Educational Modules

- Interactive, gamified learning content tailored to financial literacy topics.
- Integrated quizzes with point-based rewards.

Augmented Reality Experience

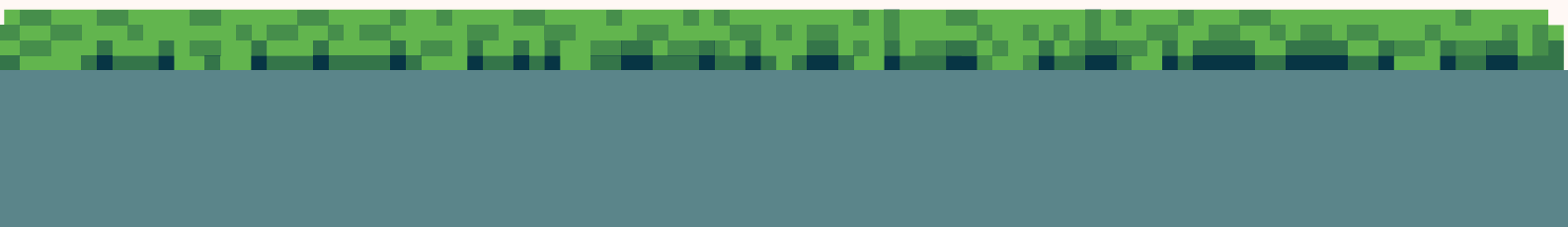
- Visualization of financial status and progress through Unity-powered AR scenes.
- Real-time interaction with assets like savings vaults or debt mountains via mobile devices.

Cross-Platform Support

- Web access via React.js.
- Mobile support via React Native and Unity AR Foundation for Android and iOS.

Security & Compliance

- Implementation of secure authentication, encryption, and anonymization protocols.
- Compliance with data protection regulations (e.g., POPIA, GDPR).

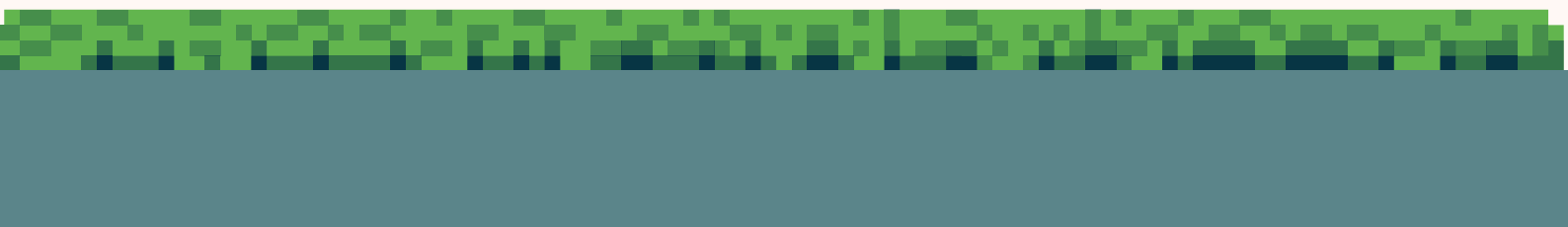


Unit, Integration and E2E testing policy

To ensure reliability across the app's modular architecture—from backend game logic and PostgreSQL triggers to frontend state management and user flows—this section defines the testing policies applied at each layer of the system. All tests are automated and integrated into the CI/CD pipeline, enabling rapid feedback and consistent quality assurance. This approach supports continuous delivery while maintaining high standards for correctness, performance, and user experience.

The backend architecture, built on Express and Socket.IO, handles real-time game events, financial simulations, and secure user interactions. These services are tested using Jest to validate core logic, lifecycle hooks, and edge-case scenarios such as score recalculations, leaderboard updates, and risk engine triggers. Integration tests ensure that modules communicate correctly—especially across authentication flows, preference persistence, and API-driven state changes. Each test suite is designed to be modular, reproducible, and environment-agnostic, supporting both local and containerized execution.

On the frontend, Cypress is used to simulate full-stack user journeys, from onboarding and challenge completion to theme switching and report downloads. These end-to-end tests validate that UI components respond correctly to backend events, maintain state across sessions, and deliver a smooth experience across devices. By embedding these tests into the CI/CD pipeline, every code change is vetted against real-world scenarios before deployment—ensuring that regressions are caught early and that the platform remains stable, engaging, and scalable.



Unit Testing policy

Purpose

Unit testing serves as the foundation of our quality assurance strategy, ensuring that individual components behave as expected in isolation. This includes utility functions, API route handlers, and frontend UI elements. By validating logic at the smallest testable level, we reduce the risk of regressions, simplify debugging, and accelerate development cycles. Unit tests also act as living documentation, clarifying the intended behavior of each module for future contributors.

Tools

We use Jest as the primary testing framework for both backend and frontend logic. On the backend, Jest handles validation of Express route handlers, PostgreSQL-triggered logic, and game lifecycle utilities. On the frontend, Jest is paired with React Testing Library to test component rendering, state transitions, and user interactions. All test suites are structured to run independently, with mocking strategies applied to isolate dependencies such as database calls, socket events, and external APIs.

Scope

Unit tests are written for:

Finance logic: Score calculations, transaction simulations, reward triggers.

Game lifecycle hooks: Match creation, player state transitions, socket event handlers.

UI components: Modals, charts, preference toggles, and theme switchers.

Each test targets a single responsibility, ensuring modularity and maintainability. Complex flows are broken down into discrete units to maximize coverage and clarity.

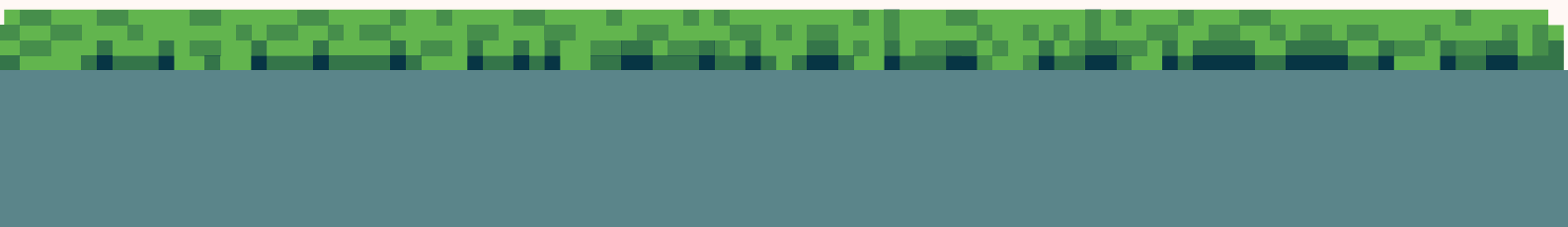
Policy

Unit tests are mandatory for all new features, bug fixes, and refactors.

Coverage threshold is set at $\geq 80\%$ per module, with exceptions requiring documented justification.

Tests are executed automatically on every pull request via the CI pipeline.

Pull requests without adequate test coverage are flagged and blocked from merging until resolved.



Integration Testing policy

Purpose

Integration testing ensures that individual modules and services interact correctly across the system. In a gamified finance environment, this includes validating the flow between authentication, real-time game logic, backend APIs, and frontend state updates. These tests help catch issues that unit tests may miss—such as mismatched data formats, socket misfires, or broken preference sync—by simulating realistic service-to-service communication.

Tools

We use Jest with Supertest for backend integration testing, allowing us to simulate HTTP requests and validate middleware, route handlers, and database interactions. For frontend integration, Cypress is used to test UI components in conjunction with backend APIs, ensuring that user actions trigger the correct server responses and state transitions. Mock servers and Dockerized test environments are employed to isolate external dependencies and maintain reproducibility.

Scope

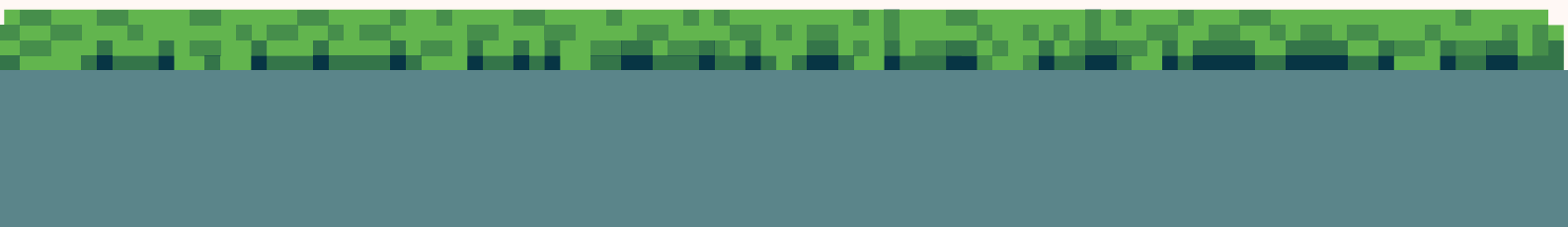
Integration tests cover:

- Authentication flows: MFA setup, session persistence, protected route access.
- Game logic: Real-time socket events, leaderboard updates, match lifecycle transitions.
- Preference management: Dark mode toggles, localStorage sync, API-driven theme persistence.
- Finance engine: Transaction simulations, risk scoring, reward triggers.

Each test suite is designed to validate cross-module behavior, ensuring that services remain decoupled yet interoperable. Tests are structured to run in parallel where possible, reducing CI latency while maintaining coverage depth.

Policy

- Integration tests are required for all features involving multiple services or modules.
- Tests must simulate realistic user flows and backend interactions.
- All integration tests are executed automatically in the CI pipeline after unit tests pass.
- Failures in integration tests block merges to the main branch and trigger alerts for review.
- Test environments must mirror production configurations as closely as possible, including database schemas, socket namespaces, and auth middleware.



E2E testing policy

Purpose

End-to-End testing validates the entire user journey across the full stack—from frontend interactions and backend responses to database updates and real-time socket events. In the context of a gamified finance platform, this means simulating how users onboard, complete challenges, view leaderboards, switch themes, and interact with financial simulations. E2E tests ensure that all layers of the system work together seamlessly, delivering a consistent and intuitive experience across devices and browsers.

Tools

We use Cypress as the primary framework for E2E testing. Cypress provides fast, reliable browser automation with deep visibility into UI behavior, network requests, and DOM state. It integrates smoothly with our CI/CD pipeline, allowing tests to run automatically on every push to staging or main branches. Screenshots and video recordings are captured for failed test runs, enabling rapid debugging and reviewer-friendly traceability.

Scope

E2E tests cover:

- Authentication flows: Signup, login, Token verification, session persistence.
- Game interactions: Starting challenges, submitting answers, viewing scores and leaderboards.
- Finance simulations: Triggering transactions, receiving feedback, unlocking rewards.
- Preference management: Dark mode toggling, theme persistence across sessions.
- Download UX: Exporting reports, viewing feedback spinners, handling errors gracefully.

Each test scenario is designed to mimic real user behavior, including edge cases such as network delays, invalid inputs, and session timeouts. Tests are written to be modular and reusable, with clear separation between setup, action, and assertion phases.

Policy

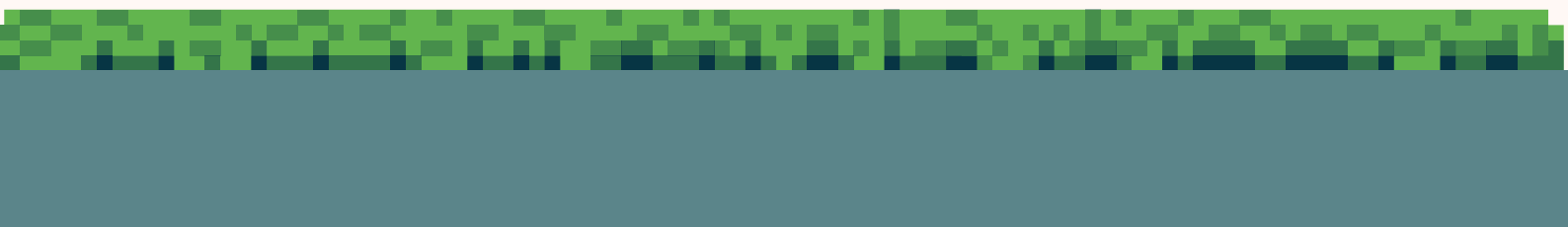
E2E tests are mandatory for all critical user flows and high-impact features.

Tests must be written before merging to main and reviewed alongside feature code.

All E2E tests are executed automatically in the CI/CD pipeline as the final gate before deployment.

Failures block production deployment and trigger alerts for immediate resolution.

Test data is reset between runs to ensure consistency and avoid state leakage.



Testing workflow

1. Development Stage

Action: Developer writes new feature or bug fix.

Output: Source code + corresponding unit tests.

Note: Code must meet modularity and documentation standards.

2. Unit Testing

Tool: Jest

Trigger: CI pipeline runs unit tests on pull request.

Validation:

- Finance logic (e.g., score calculations)

- Game lifecycle hooks

- UI components (e.g., modals, toggles)

Outcome: If tests pass → proceed; if fail → fix and retry.

3. Integration Testing

Tools: Jest + Supertest (backend), Cypress (frontend)

Trigger: CI runs integration tests after unit tests succeed.

Validation:

- Auth flows (MFA, sessions)

- Socket event handling

- API + frontend sync

Outcome: If tests pass → proceed; if fail → isolate and resolve.

4. End-to-End (E2E) Testing

Tool: Cypress

Trigger: Final CI gate before merge to main or deploy.

Validation:

- Full user journeys (login → challenge → leaderboard)

- Theme persistence

- Download UX

Outcome: If tests pass → merge allowed; if fail → blocked until resolved.

5. Deployment

Trigger: Merge to main branch.

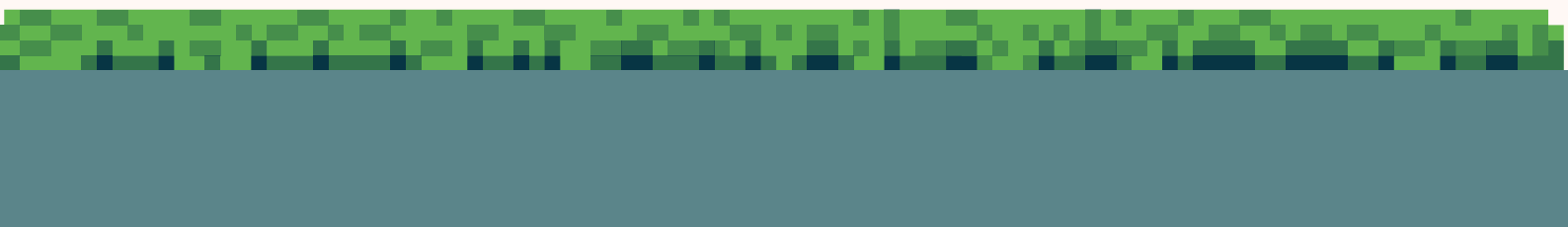
Action: CI/CD pipeline deploys to production or staging.

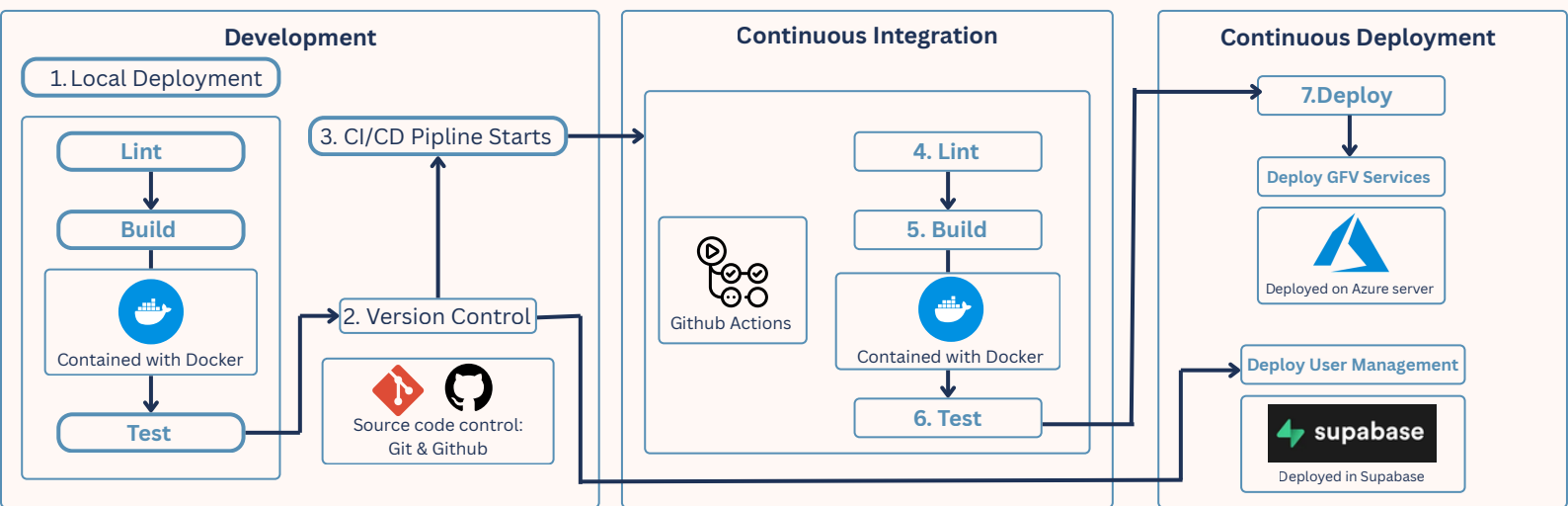
Post-Deploy Checks:

- Smoke tests

- Monitoring dashboards

- Feedback loop from QA



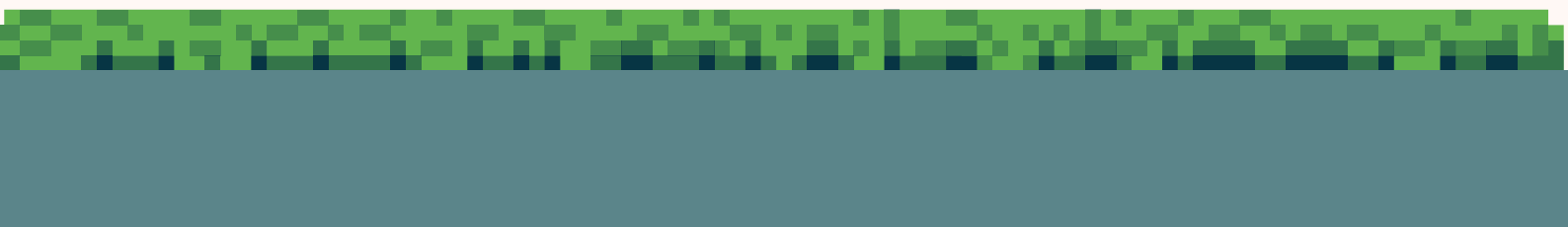


Quality Requirements testing

To ensure the gamified finance platform delivers a seamless, engaging, and resilient experience, quality testing is conducted across three critical dimensions: Usability, Performance, and Scalability. These dimensions reflect not just how the system functions, but how it feels, responds, and evolves under real-world conditions. Unlike traditional correctness testing, which focuses on whether features work as intended, quality testing probes deeper—into how users perceive the platform, how it handles stress, and how gracefully it grows.

Usability testing centres on the human experience. It evaluates how intuitive the interface is, how easily users can navigate financial challenges, and how accessible the platform remains across devices and demographics. This includes testing theme contrast for dark mode, onboarding clarity, and feedback mechanisms like spinners and alerts. The goal is to ensure that every interaction feels purposeful, frictionless, and rewarding—especially for users who may be new to financial tools or gamified environments.

Performance and scalability testing push the system to its limits. Performance testing simulates high-traffic conditions, rapid socket events, and concurrent API calls to measure latency, throughput, and error resilience. Scalability testing goes further, validating how the platform behaves as user volume, data complexity, and feature sets expand. Whether it's stress-testing PostgreSQL triggers, parallelizing scraper tasks, or autoscaling instances, these tests ensure that the app remains fast, stable, and responsive—even as it grows from a prototype into a production-grade ecosystem.



Performance

Objective

Performance testing ensures the platform remains fast, responsive, and accessible across devices and network conditions. For a gamified finance app, this means validating how quickly the UI reacts to user actions, how efficiently it renders leaderboard updates, and how smoothly it handles theme toggling, animations, and real-time feedback—even during peak usage.

Tooling: Google Lighthouse

We use Google Lighthouse, an open-source auditing tool built into Chrome, to measure and optimize key frontend performance metrics. Lighthouse tests are run against staging builds and production-like environments to simulate real-world usage. Reports are version-controlled and integrated into our CI/CD pipeline to catch regressions early.

Methodology

Test Targets:

- Login and registration flows
- Leaderboard rendering and animation
- Theme toggling and dynamic UI updates
- Report downloads triggers and modal interactions

Scenarios Simulated:

- Cold and warm page loads across devices
- Slow 3G and mid-tier CPU throttling
- Repeated navigation between dashboard and challenge views
- Accessibility checks for keyboard navigation and screen readers

Metrics Captured:

- MetricDescription
- First Contentful Paint
- Time until first visible element appears
- Largest Contentful Paint
- Time until main content is fully rendered
- Time to Interactive
- When the page becomes usable
- Total Blocking Time
- Time spent waiting on long tasks
- Speed Index
- Visual progress of page load
- Accessibility Score
- Compliance with WCAG standards
- Artifacts & Reporting

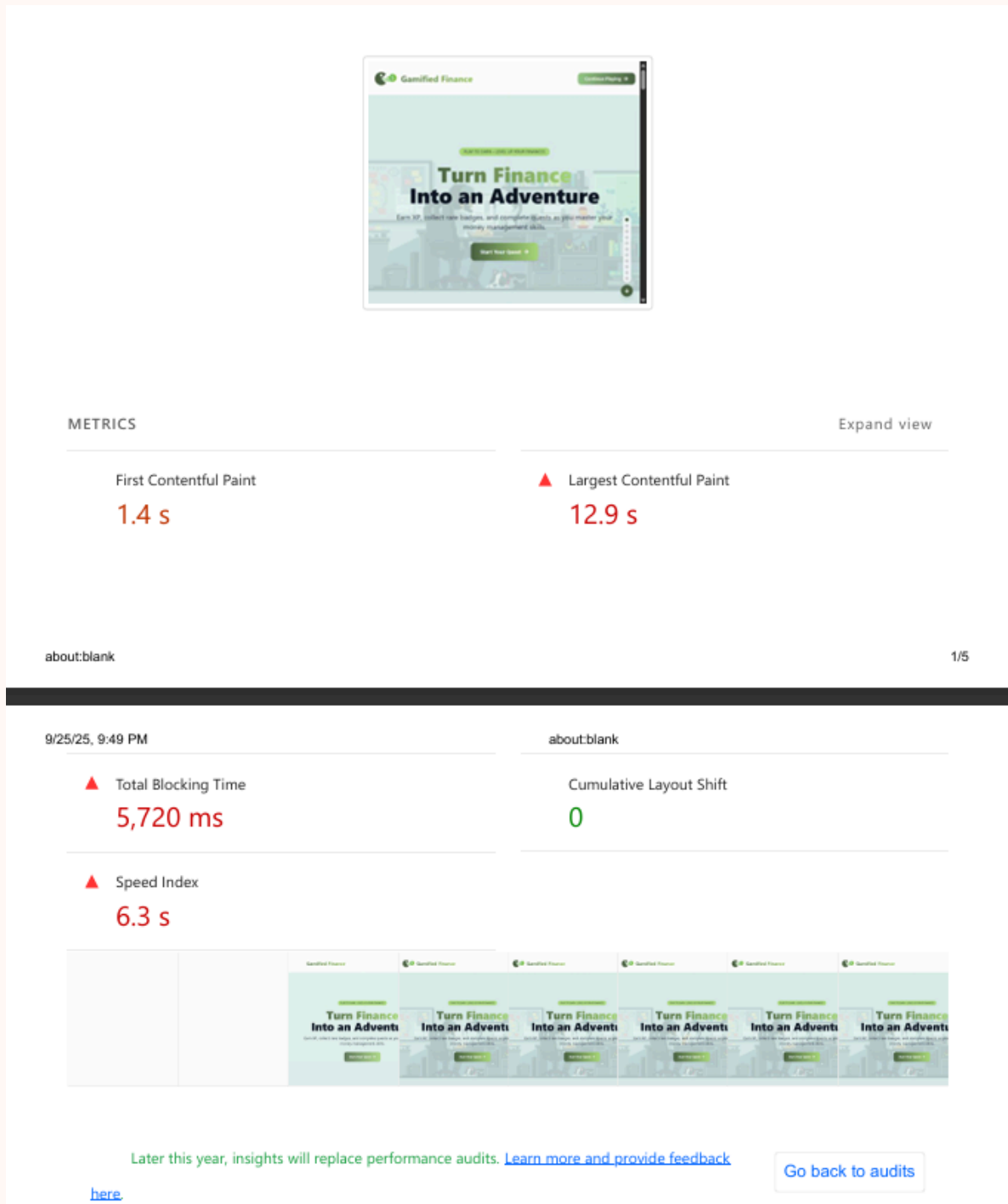


Performance

Actionable & Optimizations

- Lazy-loaded leaderboard components to reduce initial paint time
- Reduced bundle size via tree-shaking and dynamic imports
- Preloaded fonts and critical assets for faster rendering
- Improved ARIA roles and contrast ratios for accessibility
- Deferred non-critical scripts to minimize blocking time

LightHouse Metrics



Performance



METRICS

Expand view

First Contentful Paint
1.4 s
about:blank

▲ Largest Contentful Paint
7.4 s

1/4

9/25/25, 9:54 PM

about:blank

▲ Total Blocking Time
6,570 ms

Cumulative Layout Shift
0

▲ Speed Index
9.4 s



Performance



about:blank

1/5

9/25/25, 9:59 PM

about:blank

METRICS

Expand view

First Contentful Paint	▲ Largest Contentful Paint
1.0 s	19.8 s
▲ Total Blocking Time	Cumulative Layout Shift
5,580 ms	0.002
▲ Speed Index	
6.4 s	

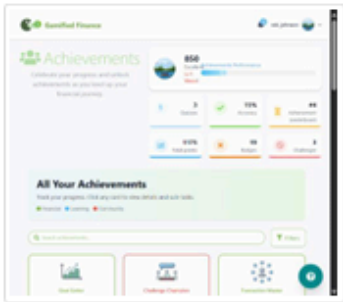


Performance

Performance

Values are estimated and may vary. The [performance score is calculated](#) directly from these metrics. [See calculator.](#)

▲ 0–49 50–89 90–100



METRICS

Expand view

First Contentful Paint
1.0 s

▲ Largest Contentful Paint
12.2 s

about:blank

1/5

9/25/25, 10:04 PM

about:blank

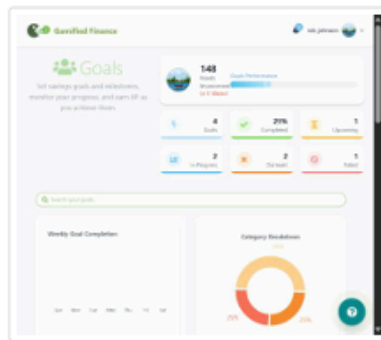
▲ Total Blocking Time
6,070 ms

Cumulative Layout Shift
0.057

▲ Speed Index
6.1 s



Performance



METRICS

Expand view

First Contentful Paint

1.3 s

▲ Largest Contentful Paint

10.1 s

about:blank

1/5

9/25/25, 10:14 PM

about:blank

▲ Total Blocking Time

5,750 ms

Cumulative Layout Shift

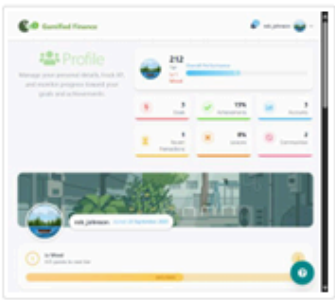
0.01

▲ Speed Index

6.1 s



Performance



METRICS

Expand view

First Contentful Paint
0.7 s

▲ Largest Contentful Paint
10.5 s

about:blank

1/4

9/25/25, 10:06 PM

about:blank

▲ Total Blocking Time
5,940 ms

Cumulative Layout Shift
0.114

▲ Speed Index
4.5 s

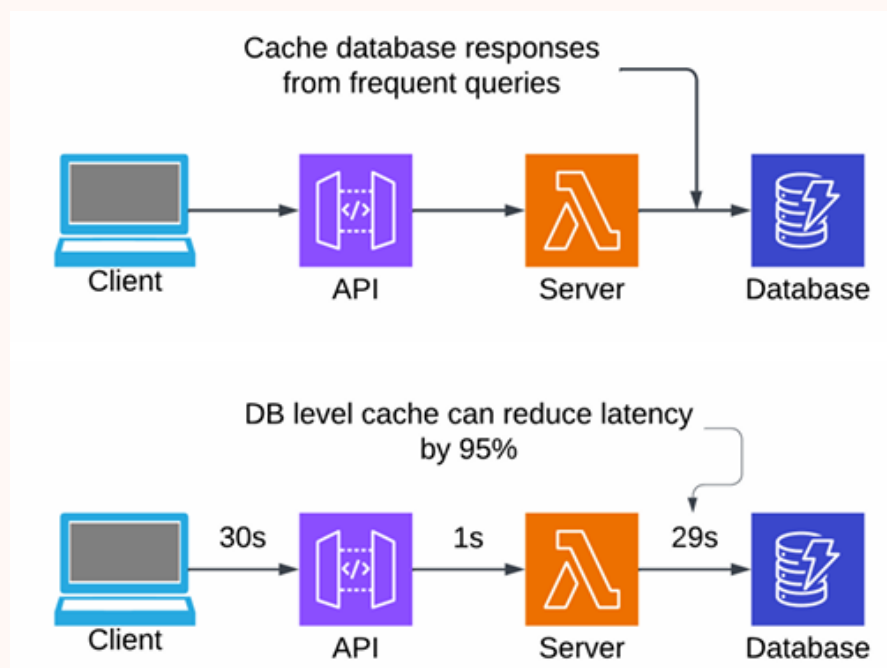


Performance

Insights

The performance audits revealed key behavioural patterns across the platform:

- Cold vs Warm Load Disparity:
 - On initial page load, Lighthouse scores were lower due to unprimed assets and uncached data. However, subsequent loads showed significant improvements in FCP, LCP, and TTI—confirming that Redis caching is effectively reducing backend latency and frontend render time.
- Read-Heavy Architecture:
 - Analysis of endpoint traffic and database queries indicated a strong bias toward read operations. This validates the decision to optimize for fast data retrieval, with Redis acting as a high-speed layer for leaderboard data, preference states, and simulation results.
- Caching Impact:
 - Redis cache hit rates correlated with improved Lighthouse performance scores, especially on dashboard and leaderboard views. This reinforces the importance of preloading and caching strategies for user-facing components.
- Frontend Bottlenecks:
 - Minor delays were observed in rendering animated components during cold loads. These were mitigated by lazy-loading non-critical assets and deferring heavy scripts until after interaction readiness.



credit: @ 2025 Arjay McCandless

Availability

Objective

Availability testing ensures that the platform remains consistently reachable, responsive, and fault-tolerant across all services. For a gamified finance app, this means validating that authentication, leaderboard updates, financial simulations, and real-time socket events are always accessible—even during deployments, network fluctuations, or backend failures.

Tooling: Uptime Kuma

We use Uptime Kuma as a lightweight, self-hosted monitoring tool to track the availability of critical endpoints and services. It performs periodic health checks via HTTP, TCP, and WebSocket protocols, and triggers alerts on downtime or latency spikes. Grafana complements this by visualizing uptime trends, alert history, and service health metrics.

Methodology

Monitored Targets:

- API endpoints for login, registration, transactions, and city retrieval
- Background services like report generation and preference syncing
- Database and cache layers (PostgreSQL, Redis)

Check Types:

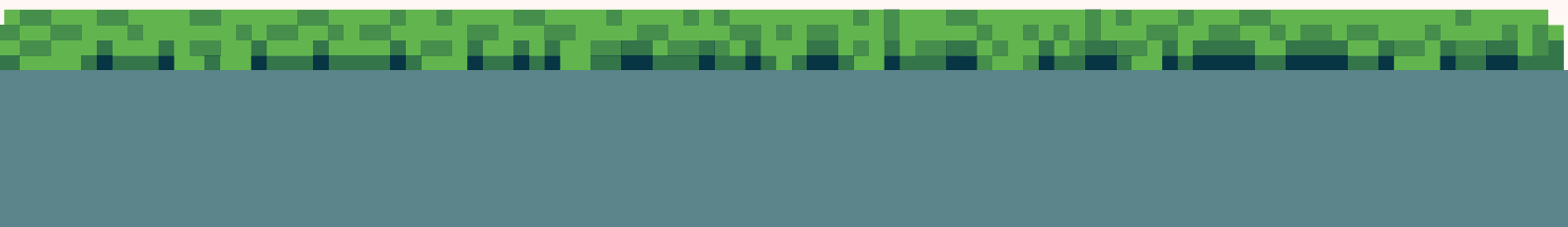
- HTTP status code validation (expecting 200/201)
- Response time thresholds (e.g., <500ms)
- WebSocket handshake and heartbeat checks
- TCP port availability for internal services

Artifacts & Reporting

- Uptime Kuma dashboards show availability percentages, response times, and downtime logs
- Grafana panels visualize historical uptime, latency trends, and alert frequency
- Weekly uptime reports are exported for reviewer analysis and SLA tracking
-

Actionable & Optimizations

- Added redundant health checks for critical endpoints across multiple regions
- Implemented graceful degradation for leaderboard and report services
- Refactored socket logic to auto-reconnect and retry on disconnect
- Tuned reverse proxy timeouts and load balancer health probes
- Introduced failover strategies for Redis and PostgreSQL clusters



Availability with Uptime Kuma



Actionable & Follow-ups

The uptime graphs confirm consistent availability across critical endpoints such as community, transactions and the dashboard. Most services maintained >90 % uptime over the monitored period, with no recorded downtime or latency spikes exceeding 500ms. This validates the platform's resilience under normal and peak usage conditions."

To further enhance reliability:

Database Redundancy:

- We've implemented a snapshot-based redundancy strategy for PostgreSQL, storing periodic snapshots of the database to ensure rapid recovery in case of failure. This allows us to restore state with minimal data loss and maintain service continuity during outages or migrations.
- Socket Resilience:
 - WebSocket connections now include auto-reconnect logic and heartbeat validation to reduce disconnects and improve real-time reliability.
- Service Isolation:
 - Background services such as report generation and preference syncing are decoupled from core gameplay flows, allowing graceful degradation during partial outages.

Usability

1. Testing Approach

We used a remote usability testing approach with participants completing structured tasks while answering a follow-up questionnaire. Data was collected through Google Forms, quantitative Likert scale responses (1–5), and qualitative open-ended feedback.

1.1 Task Design

Participants were asked to:

- Navigate key pages (Home, Accounts, Goals, Learn, Achievements, Community, Support, Profile).
- Perform representative tasks such as viewing goals, checking accounts, and exploring achievements.
- Provide feedback on clarity, navigation, and gamification elements.

1.2 Process Followed

- Participants accessed the system remotely.
- They were guided through a series of tasks.
- After task completion, they filled in the Google Form survey.
- Responses were analyzed to identify usability strengths, challenges, and areas for improvement.

Link to the questionnaire:

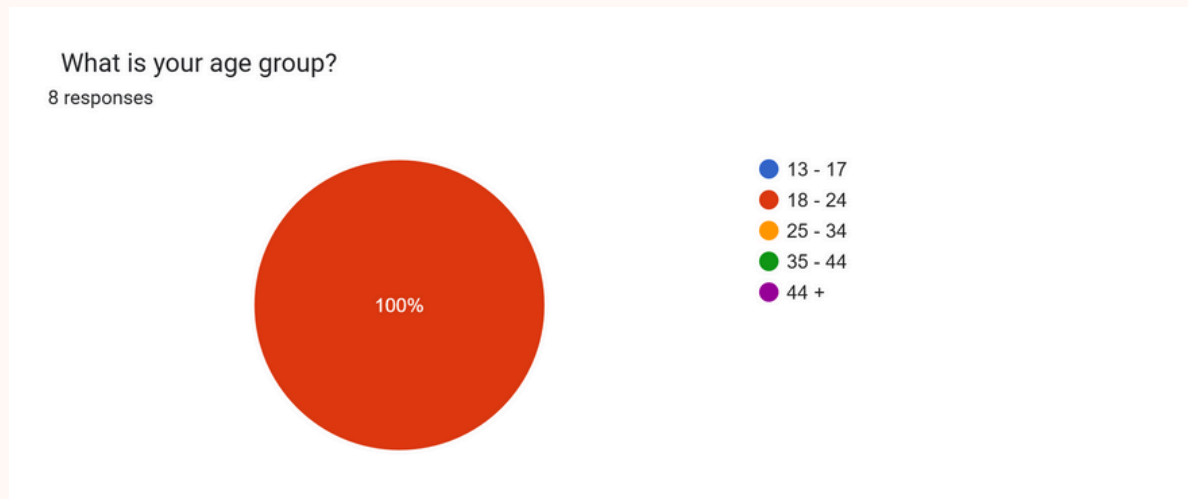
<https://docs.google.com/forms/d/e/1FAIpQLSfwSvnjamtUBBULik72Vvyt3pTPmWAtWtLtm09L3h7MgzOfQg/viewform?usp=header>

2. Usability Testing Results

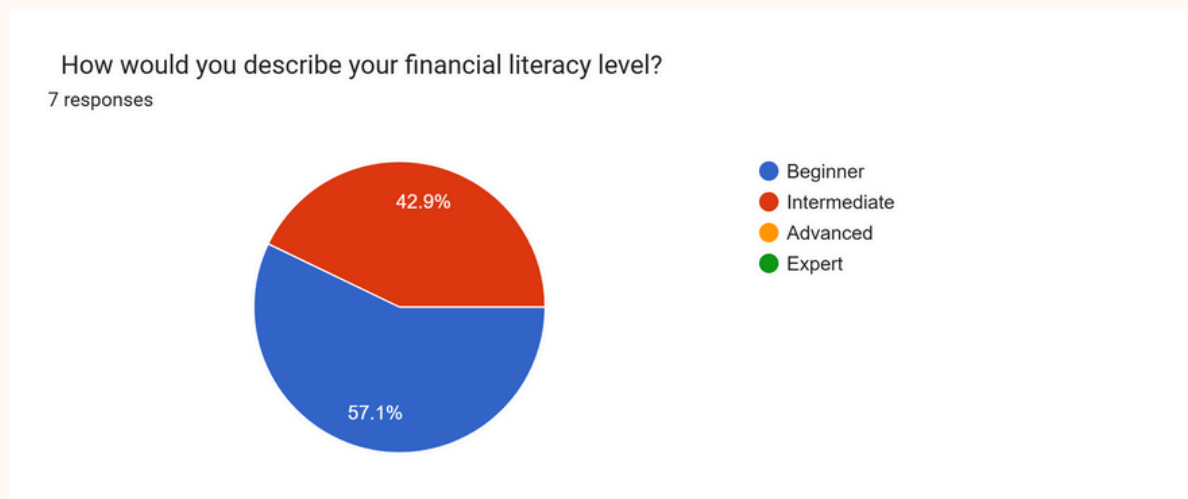
2.1 Demographic Information

- Age Groups:
 - 100% (8/8) of participants were between 18–24.
- Financial Literacy Levels:
 - Beginner: 4 (57.1%)
 - Intermediate: 3 (42.9%)
 - Advanced/Expert: 0

Age distribution

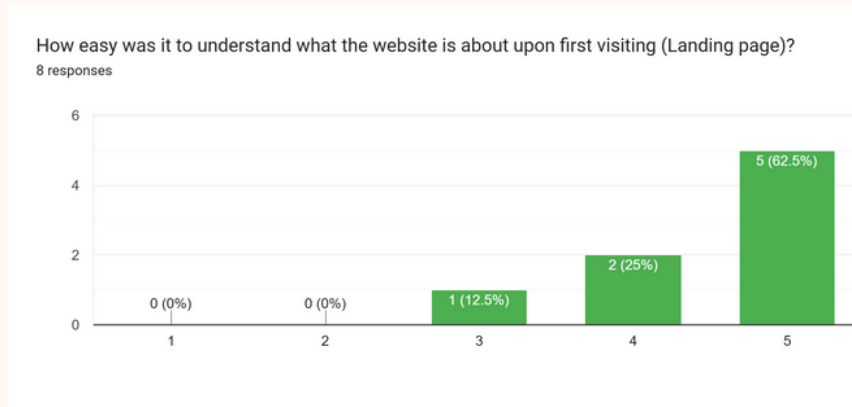


Financial literacy distribution

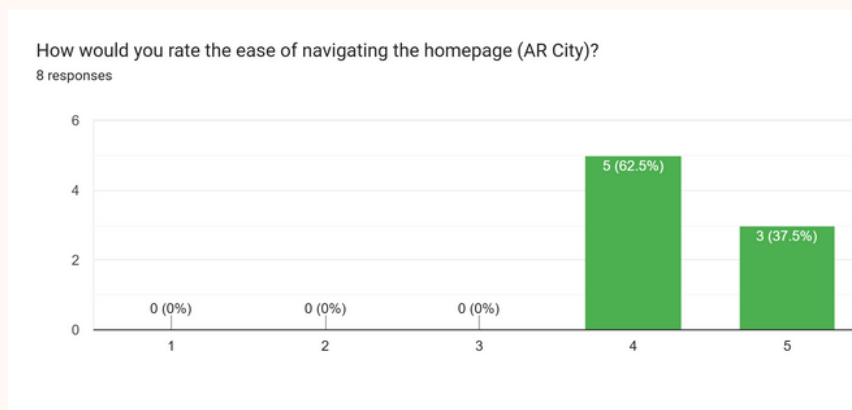


2.2 First Impressions & Navigation

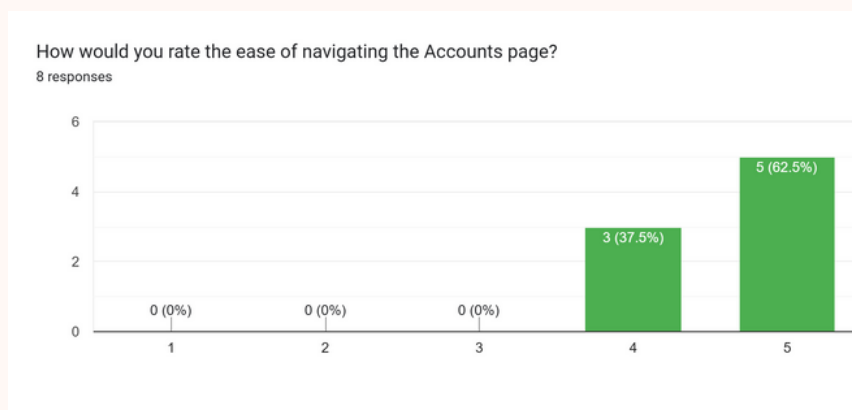
- Landing Page Understanding:
 - Majority rated 4–5 (87.5%), with 62.5% giving it a perfect 5.



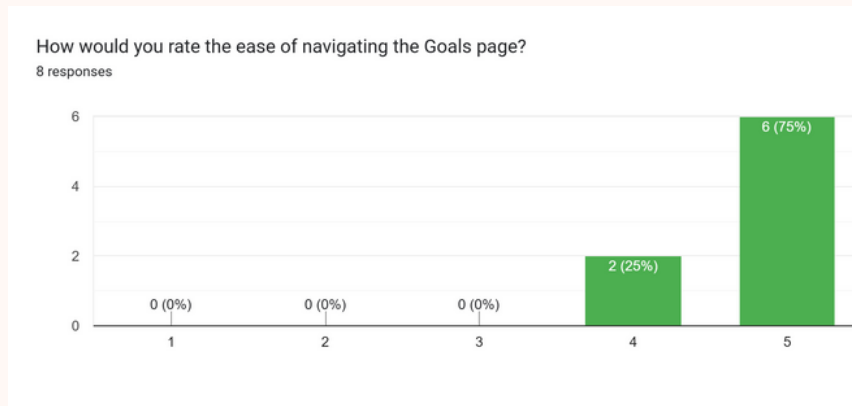
- Homepage (AR City) Navigation:
 - 100% rated navigation as 4 or 5.
 - Positive feedback: “The labelled buildings providing summaries are amazing.”
 - Issues: Some buttons didn’t work; WiFi limited experience.



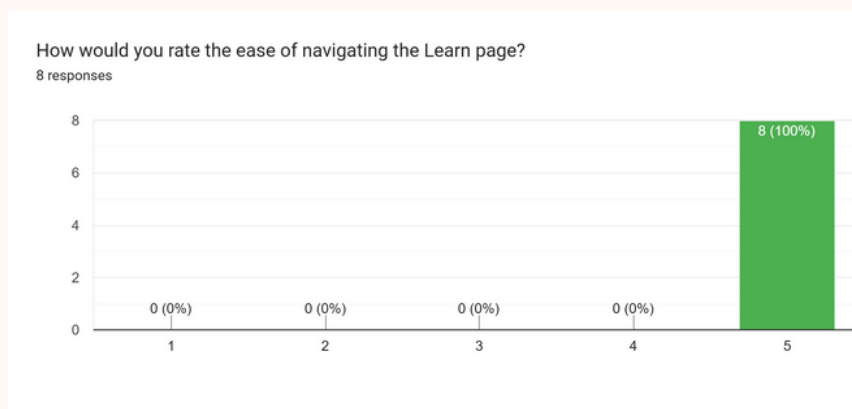
- Accounts Page:
 - 100% rated navigation as 4 or 5.
 - Feedback: Mostly functional, minor styling issues.



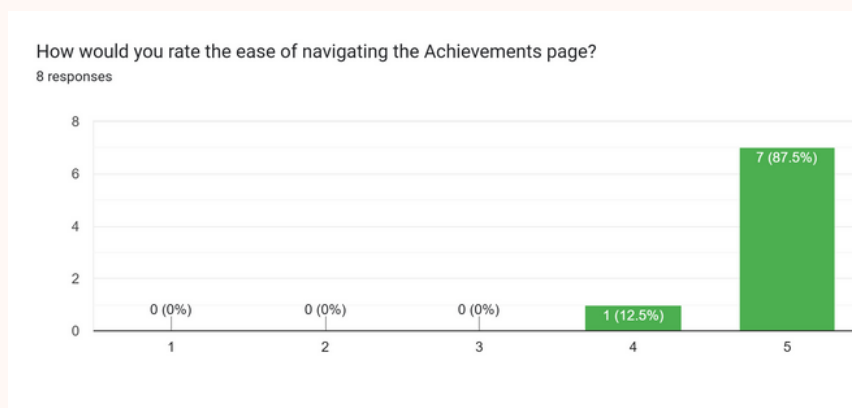
- Goals Page:
 - 75% gave it 5, 25% gave it 4.
 - Suggestions: Add heading for completed goals, reposition recently completed goals.



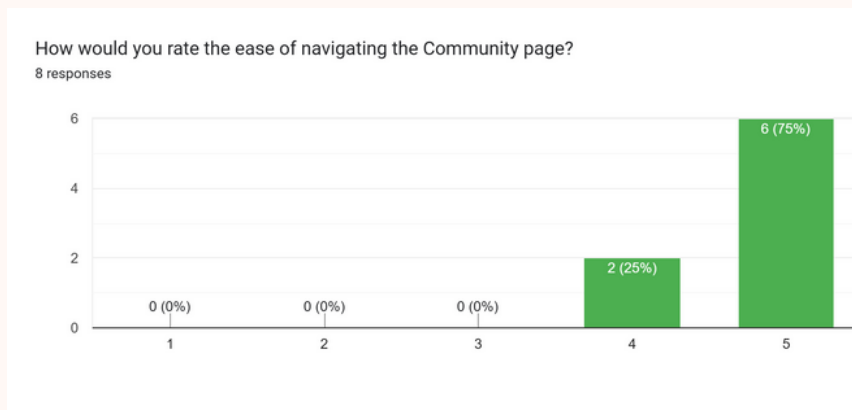
- Learn Page:
 - 100% gave it a perfect 5.
 - Feedback: Works well, but descriptions should truncate properly.



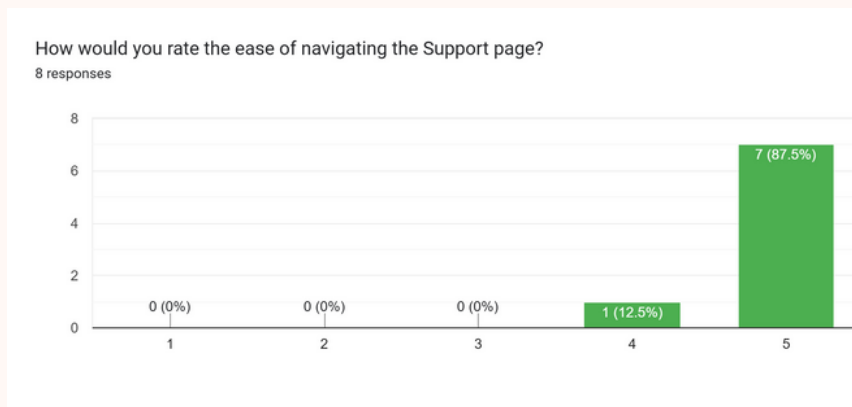
- Achievements Page:
 - 87.5% gave it 5.
 - Suggestion: Enable click-through for more achievement info.



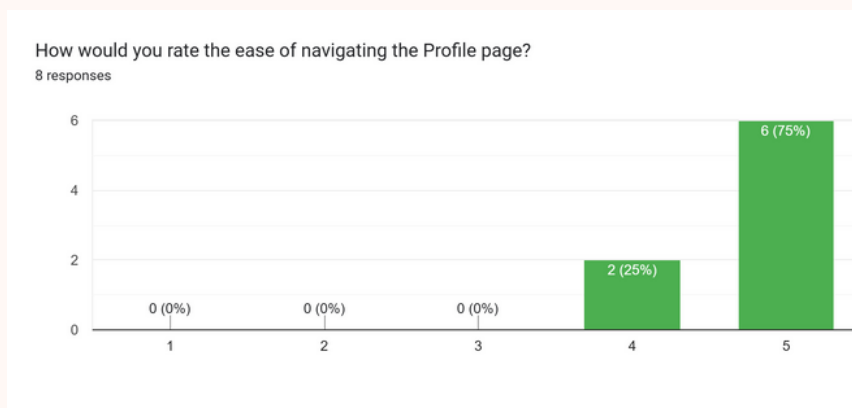
- Community Page:
 - 75% gave it 5.
 - Issues: Dice not rolling in the game, delete not automatic, dark mode text readability issues.



- Support Page:
 - 87.5% gave it 5.
 - Mostly functional.

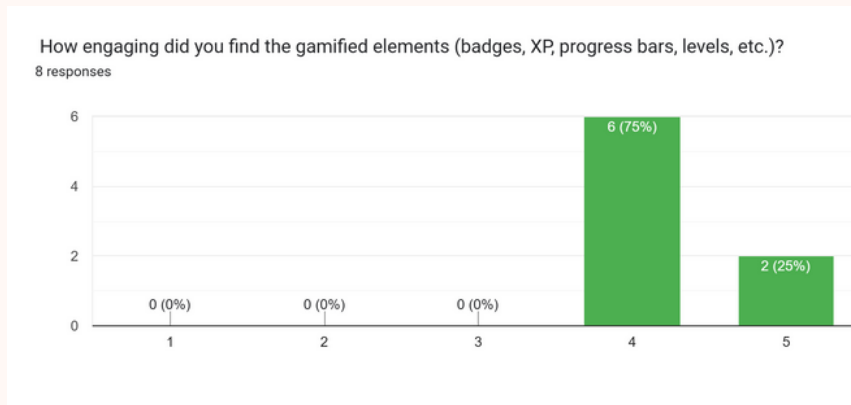


- Profile Page:
 - 75% gave it 5, 25% gave it 4.

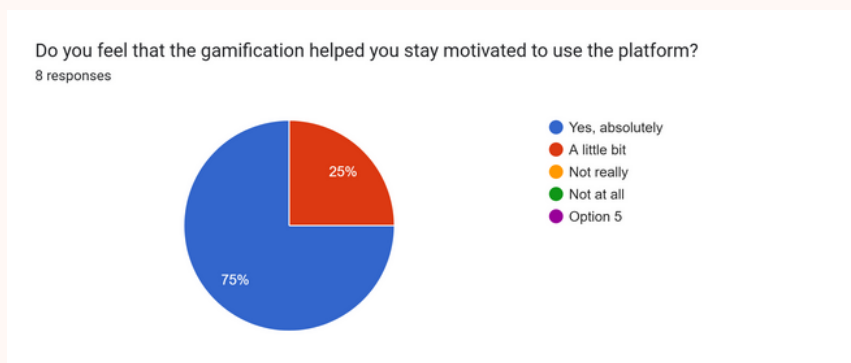


2.3 Engagement & Gamification

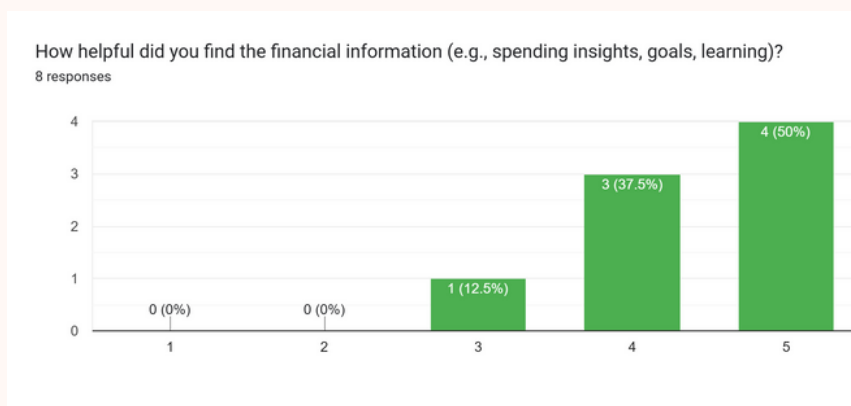
- Gamified Elements (XP, Badges, Progress):
 - 75% rated as engaging (4), 25% rated as very engaging (5).



- Motivation from Gamification:
 - 75%: Yes, absolutely.
 - 25%: A little bit.

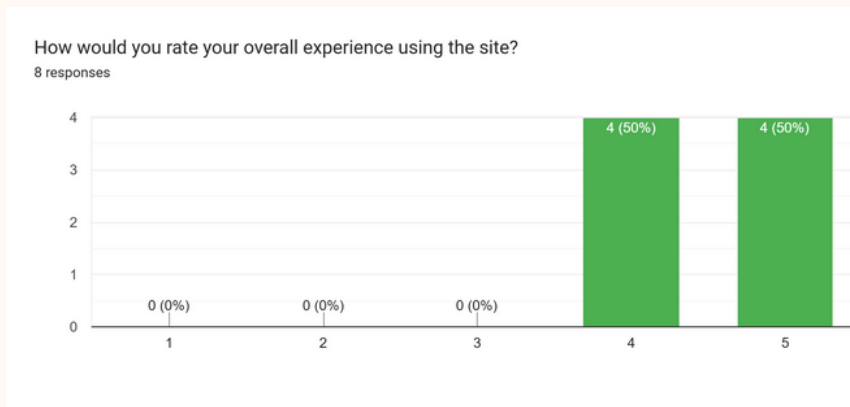


- Confusion with Game Features:
 - None reported.

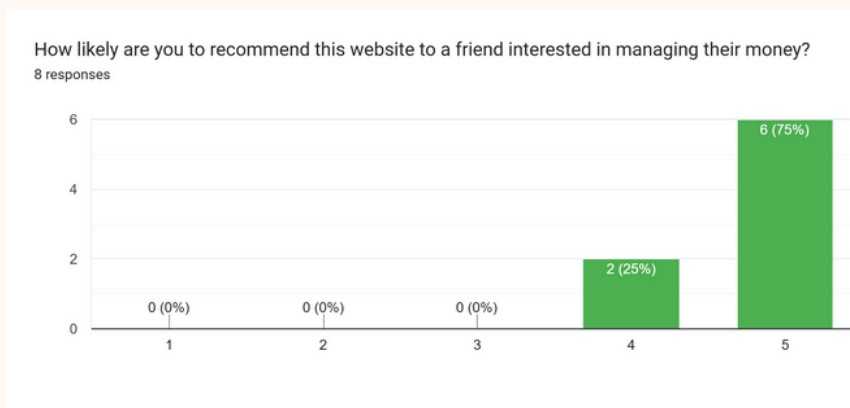


2.4 Overall Experience

- Overall Satisfaction:
 - 50% rated 4, 50% rated 5.



- Likelihood to Recommend:
 - 75% “Very likely” (5), 25% “Likely” (4).



Question: What did you enjoy most about the website?

- Most Enjoyed Features:
 - AR City, gamified experience, monopoly-style game, insights.

Question: What do you think could be improved?

- Areas for Improvement:
 - Tabs should auto-display info.
 - Dark mode readability.
 - More detail on achievements.
 - Styling refinements.

3.Scenarios

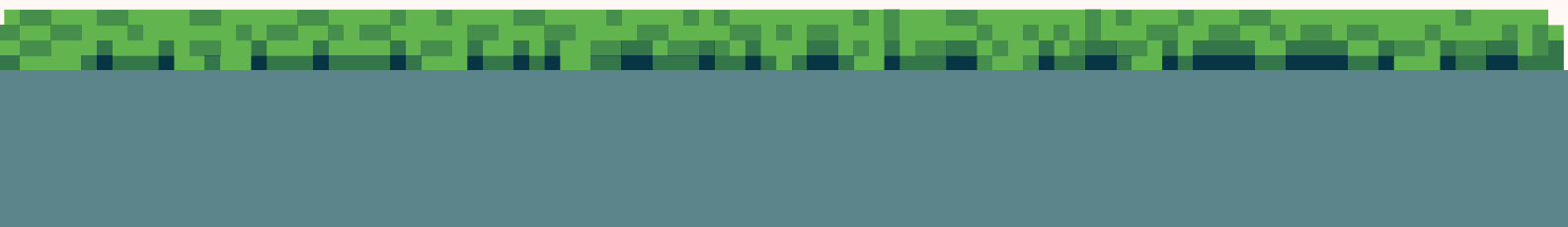
- **Example Scenarios tested:**
 - A user logs in and checks their account balances.
 - A user sets a new financial goal and checks progress.
 - A user explores achievements and attempts to unlock one.
 - A user engages with the community by checking challenges.
 - A user browses the Learn module and attempts a quiz.

4.Key Areas for Improvement

- **Navigation:** Improve clarity of completed goals, add headings.
- **Achievements:** Enable click-through details.
- **Community:** Improve dice/game function, add loading page, fix dark mode readability.
- **UI Styling:** Minor alignment and content truncation issues.

5.Solutions

- Introduce progressive disclosure for goals.
- Add achievement detail modals.
- Refine dark mode CSS for contrast.
- Improve loading states for community and support.



Gamified Finance
In Collaboration With

